

Learning Objectives

The primary objective remains to practice the development workflow

edit (and save) → compile → run

assuming compilation succeeds.

Closely related to this is becoming comfortable with the Unix command line. Throughout the course, the shell is used to organize files, invoke the compiler, execute programs, and automate repetitive tasks. Although the shell itself is only used briefly in this worksheet, becoming fluent with it is an important part of developing an efficient programming workflow.

The secondary objective is to develop an intuition for how programs are executed. In particular, this worksheet introduces the following concepts:

- the lifetime of local and global variables,
- parameter passing using the call-by-value principle,
- control structures (`if`, `while`, `do...while`, and `for`),
- and the role of the stack in implementing function calls and local variables.

As in the previous worksheets, these concepts are introduced through small experiments with the ABC programming language rather than by presenting their formal implementation immediately. The underlying implementation will later be studied in the Bottom-Up part of the course.

Step 1

Throughout the rest of the course, you will create a growing collection of source files, worksheets, and experiments. It is therefore worth spending a few minutes organizing your files from the beginning.

Tasks

Use the shell to create a directory structure for the course. For example, create the following directories inside your home directory:

```
hpc0/
├── session1/
├── session2/
└── session3/
```

Move the files from the previous sessions into their corresponding directories.

You are free to organize your files differently if you prefer. The important point is to develop the habit of keeping projects well organized.

Step 2 – Lifetime of Variables

Variables have not only a **scope** (where they can be accessed) but also a **lifetime** (when they exist).

A **global variable** has **scope** from its declaration until the end of the program. Its **lifetime**, however, begins when the program starts and ends when the program terminates.

A **local variable** has **scope** only within the block in which it is declared. Its **lifetime**, however, begins when the corresponding function is entered and ends when the function returns.

Remark: Notice that the beginning of a variable's scope and the beginning of its lifetime are not necessarily the same.

A global variable already exists when the program starts, but it cannot be accessed before its declaration. In other words, its lifetime begins before its scope.

For local variables, the situation is different. Their scope and lifetime begin at approximately the same time—when execution enters the block in which they are declared.

```
1 @ <stdio.h>
2
3 global a: int = 42;
4
5 fn foo()
6 {
7     local b: int;
8     printf("foo entered: b = %d\n", b);
9     b = a;
10    printf("foo leaving: b = %d\n\n", b);
11 }
12
13 fn main()
14 {
15     // printf("calling foo:\n");
16     foo();
17
18     // printf("calling foo:\n");
19     foo();
20
21 }
```

Tasks

1. Compile and run the program `ex3.abc` shown on the right. On my system, I obtain the following output:

```
MCL:tmp lehn$ ./a.out
foo entered: b = 1
foo leaving: b = 42

foo entered: b = 42
foo leaving: b = 42
```

Your output may differ because, as we will see, the program exhibits **undefined behavior**.

2. If we carefully trace the control flow of the program and compare it with the output, the result initially appears to contradict the statement above about the lifetime of local variables. Go through the program line by line and determine exactly when each line of output is produced.

Pay particular attention to the following:

- (A) In line 8, the value of the variable `b` is printed before `b` has been initialized. The value printed during the first call to `foo`—`1` on my system—is therefore undefined.
 - (B) In line 9, the value of `b` is overwritten with the value of `a`, which is `42`.
 - (C) During the second call to `foo`, line 8 once again prints the value of `b` before the variable has been initialized. On my system, the value `42` is printed. This is the value that `b` had at the end of the previous function call. It therefore appears as though the local variable `b` has “survived” the previous call to `foo`.
3. Uncomment the `printf` statement in line 15. Compile and run the program again. Two additional lines are now printed. More importantly, however, the output produced by `foo` should also have changed. For example, I obtain:

```
MCL:tmp lehn$ ./a.out
calling foo:
foo entered: b = 0
foo leaving: b = 42

calling foo:
foo entered: b = 0
foo leaving: b = 42
```

We can now observe that `b` did not survive the previous function call after all. Instead, a new local variable `b` was created during the second call to `foo`. It merely appears that this newly created variable was initialized to zero.

4. Compile the program again, this time with optimization enabled. For example:

```
MCL:tmp lehn$ abc -O1 ex3.abc
```

On my system, I obtain the following output:

```
calling foo:
foo entered: b = 73896
foo leaving: b = 42

calling foo:
foo entered: b = 73896
foo leaving: b = 42
```

It now appears as though `b` is initialized to `73896` during every call. However, if the `printf` statements in lines 15 and 18 are commented out again, I obtain:

```
MCL:tmp lehn$ abc -O1 ex3.abc
MCL:tmp lehn$ ./a.out
foo entered: b = -1148205632
foo leaving: b = 42

foo entered: b = 73896
foo leaving: b = 42
```

Conclusion

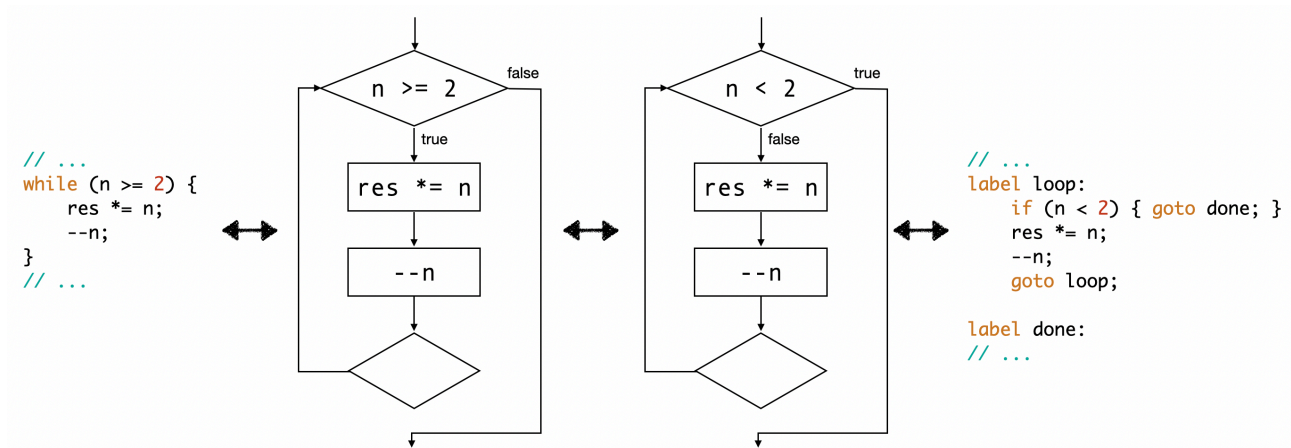
- a. Local variables do not survive a function call.
- b. Local variables are created anew for every function call and have an undefined value unless they are explicitly initialized.
- c. If an undefined value is printed, the output may depend on the date, the time of day, your current mood, and the weather.

Step 3 – Control Flow and goto

In the lecture, we saw how **flowcharts** can be used to describe the control flow within a function. The following conventions were used:

- **Conditional branches** are represented by a diamond containing the condition.
- **Unconditional branches** are represented by an empty diamond.
- All other statements are represented by rectangles containing the corresponding statement.
- The diamonds and rectangles are arranged vertically, reflecting the fact that the corresponding machine instructions are stored sequentially in memory. Arrows indicate the order in which the instructions are executed. Execution always starts at the top.

To better understand how control structures can be translated into primitive machine instructions, we also showed how a **while** loop can be implemented using **goto** statements:



Comparing the flowchart with the version using **goto** reveals two simple rules:

- An empty diamond corresponds to an unconditional **goto**.
- A diamond containing a condition corresponds to a statement of the form
`if (...) { goto ... }`

This construct is often referred to informally as a conditional **goto**.

Tasks

You will derive the rules for expressing control structures using **goto** statements by examining several examples.

1. As a first example, use the following program (**if.abc**):

```
@ <stdio.h>

fn main()
{
    local ch: int = getchar();

    if (ch == 'A') {
        printf("I got an A!\n");
    } else {
        printf("Not an A :(\n");
    }
}
```

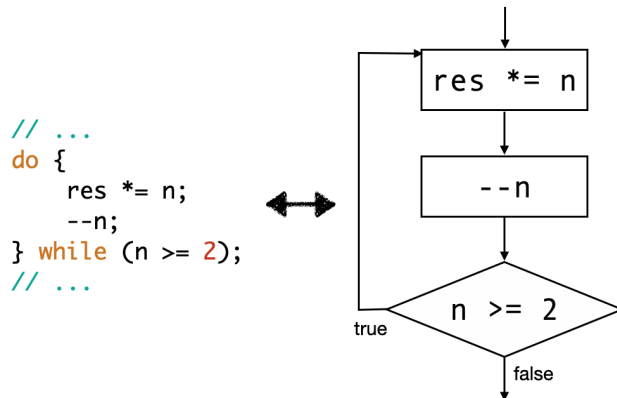
First, test the program with several different inputs.

Then:

- (a) Describe the program using a flowchart.
- (b) Transform the flowchart into a version that can be implemented using **goto** statements.
- (c) Implement the resulting control flow in a new program called **if_goto.abc**.

2. `do...while`

A `do...while` loop can be described by the following flowchart:



The following program (`do_while.abc`) repeatedly reads and prints characters until a period ('.') is encountered:

```

@ <stdio.h>

fn main()
{
    local ch: int;

    do {
        ch = getchar();
        printf("got '%c' (ASCII %d)\n", ch, ch);
    } while (ch != '.');
}
  
```

Tasks

- Compile and run the program. Verify that the terminating character ('.') is also printed.
- Describe the program using a flowchart.
- Transform the flowchart into a version that can be implemented using `goto` statements.
- Implement the resulting control flow in a new program called `do_while_goto.abc`.

3. `for`

Repeat the entire procedure for the following program (`for.abc`):

```

@ <stdio.h>

fn main()
{
    local n: int = 5;
    local res: int = 1;

    for (local i: int = 1; i <= n; ++i) {
        res *= i;
    }
    printf("res = %d\n", res);
}
  
```

That is:

- compile and run the program,
- describe its control flow using a flowchart,
- transform the flowchart into a version that can be implemented using `goto` statements,
- and implement the resulting control flow in a new program called `for_goto.abc`.